

Horizon-RIC — Full Technical Report

A runtime-assurance and evidence-binding rApp for federated AI-RAN spectrum control

490 passed · 0 failed

torch-free

reproduced from clean install

Repository: [Danielfoojunwei/PreceptualAI-Horizon-RIC](#)

Independent technical review · all quantitative results reproduced empirically

Python 3.11.15 · Linux x86-64 · no accelerator

Horizon-RIC — A Full Technical Report

A runtime-assurance and evidence-binding rApp for federated AI-RAN spectrum control

Scope. This report is a rigorous, evidence-first technical analysis of the `Danielfoojunwei/PreceptualAI-Horizon-RIC` repository. Every quantitative claim below was **reproduced from a clean install** in the environment described in §2, not copied from the README. Where a number is taken from a committed artefact, the artefact path is given and the live reproduction command is shown so the reader can re-derive it. File:line citations point at the exact mechanism under discussion.

Table of contents

- 1. Thesis and contribution
- 2. Empirical methodology and environment
- 3. Dependencies — the torch-free trust base
- 4. System architecture
- 5. The Decision Safety Shield
- 6. The evidence subsystem
- 7. The federated trust stack
- 8. The physical-layer adversarial testbed
- 9. O-RAN adapters and runtime resilience
- 10. Security subsystem
- 11. Test methodology — how the tests prove the claims
- 12. Consolidated empirical results
- 13. Honest limitations
- 14. Reproduction appendix

1. Thesis and contribution

Horizon-RIC is an **O-RAN Non-RT-RIC rApp** whose single design thesis is:

The binding constraint on deploying federated AI-RAN agents onto licensed spectrum is not decision-making capability — it is trust. When a federated deep-RL agent is poisoned, drifted, or adversarially driven, what mathematically bounds it from steering a cell out of band or over a power ceiling, and how does an operator prove to a regulator what the agent actually did?

The architecture answers this with an **"AI proposes, Shield disposes"** pattern: a neural block emits an action; a deterministic, model-independent Shield projects that action onto enumerated radio invariants; and every disposition is bound into a tamper-evident, replayable, RFC-3161-anchored evidence record.

The repository is explicit — and this is intellectually important — that it makes **exactly one** state-of-the-art novelty claim: the **decision-level evidence / audit binding** (a per-decision `SafetyCertificate` bound to a SHA-256 hash chain, an RFC-3161 timestamp, a replayable counterfactual, and model-provenance threading). Everything else (the shielding idea, the robust aggregators, the crypto primitives) is declared prior art and used as an honestly-graded baseline. This disciplined scoping is itself a research contribution: the codebase is a **benchmark that demonstrates where its own defences fail**, not a marketing surface.

2. Empirical methodology and environment

All results in this report were produced as follows:

Item	Value (measured)
Python	3.11.15 (CPython)
Platform	Linux 6.18.5, x86-64, no accelerator
Install	<code>python -m venv ... && pip install -e ".[dev,otel,oran,persistence]"</code>
Source size	16,103 LOC across <code>src/horizon_ric</code>
Test size	9,372 LOC across <code>tests/</code>
Python modules	162 <code>.py</code> files
Fast test pack	490 passed, 3 skipped, 1 xfail, 0 failed (<code>pytest -m "not integration and not slow"</code> , exit 0)

The `pip install -e .` on the stock host required a clean virtualenv only because the base image ships a Debian-managed `cryptography` ; in an isolated environment the install completes with no compilation of native ML wheels — consistent with the torch-free claim.

Reproducibility discipline. The headline benchmarks are deterministic (seeded numpy RNGs). This report treats determinism as a *testable property*: §12 shows a live re-run of `poisoning_shield_benchmark.py` producing numbers that match the committed `benchmarks/results/poisoning_shield.json` **to all printed significant figures** (e.g. Krum baseline distance `12.196867167126614` , Fang-median FedAvg distance `8.233744329869157`). A benchmark that reproduces bit-for-bit is a benchmark a regulator can audit.

3. Dependencies — the torch-free trust base

`pyproject.toml` declares a deliberately narrow runtime dependency set. The design rationale is stated in a source comment (`pyproject.toml:29-32`): *"The Shield validates the decisions a neural-PHY block emits ... it never runs a neural network itself. Everything here installs on a stock Python 3.10+ with no CUDA / no accelerator."*

Runtime dependencies (versions as resolved in this environment):

Package	Resolved	Role in the codebase
<code>numpy</code> 2.4.6	linear algebra	the entire PHY testbed, aggregators, DSA env — no torch
<code>pydantic</code> 2.13.4	schema validation	<code>evidence/schema.py</code> <code>DecisionRecord</code> and all wire models
<code>fastapi</code> 0.139.2/ <code>starlette</code> 1.3.1	REST surface	<code>rapp/api.py</code> , dashboard, health endpoints
<code>httpx</code> 0.28.1	async HTTP	A1/R1 wire adapters (behind circuit breakers)
<code>cryptography</code> 49.0.0	RSA-PSS, PSS/OAEP	<code>security/hsm.py</code> , <code>provenance/signing.py</code>
<code>rfc3161-client</code> 1.0.7	RFC-3161 TSA	<code>evidence/rfc3161.py</code> (Rust ASN.1 parser, no hand-rolled ASN.1)
<code>pybreaker</code> 1.4.1	circuit breaking	<code>runtime/circuit_breaker.py</code>
<code>casbin</code> 1.43.0	RBAC	<code>security/rbac.py</code> "RBAC with domains"
<code>python-jose[cryptography]</code> 3.5.0	RS256 JWT	<code>security/jwt.py</code>
<code>authlib</code> 1.7.2	OAuth2 client	<code>rapp/auth.py</code>
<code>sqlalchemy</code> 2.0.51	evidence persistence	<code>evidence/store.py</code> <code>SqliteEvidenceStore</code>
<code>structlog</code> 26.1.0/ <code>prometheus-client</code> 0.25.0	observability	structured logs + <code>/metrics</code>

Optional extras wire the real southbound stack: `oran` (`ncclient` 0.7.1, `pyang` 2.7.1, `asn1tools` — NETCONF/YANG for O1, ASN.1 for A1/E2), `otel` (OpenTelemetry 1.44), `persistence` (`psycpg`), and `dev` (`pytest` , `ruff` , `mypy` , `bandit` , and `z3-solver` 5.0 — used to *prove RBAC policy completeness with an SMT solver*, `tests/test_rbac_smt_completeness.py`).

A CycloneDX SBOM (`deploy/sbom/horizon-ric-sbom.json` , **67 components**) is generated and cosign-attested in CI (`.github/workflows/build.yml`), giving the supply-chain provenance O-RAN WG11 asks for.

Why this matters analytically. Keeping torch off the real-time path is not a convenience — it is what makes the *entire adversarial-ML story auditable at code level*. Because the neural receiver's gradient is hand-derived numpy (§8), it can be checked against finite differences

(§11.1); there is no "the attack only works because of a framework quirk" objection available to a skeptic.

Figures — System topology and architecture

Figure 1. Deployment topology. Where the Horizon-RIC rApp sits in the O-RAN control hierarchy (SMO / Non-RT RIC), its R1/A1/O1 interfaces, the E2 telemetry it grades decisions against, the external RFC-3161 timestamp authorities and HSM it anchors trust to, and the federated DSA client fleet whose shared policy it wraps.

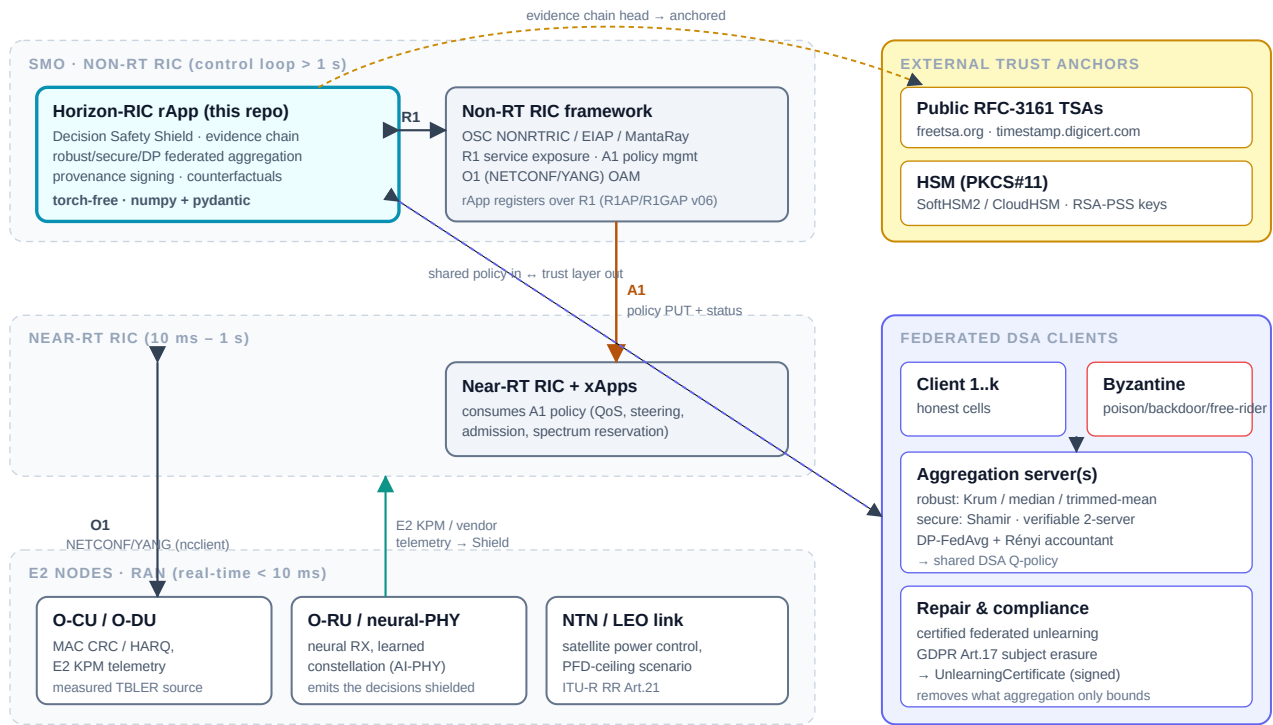
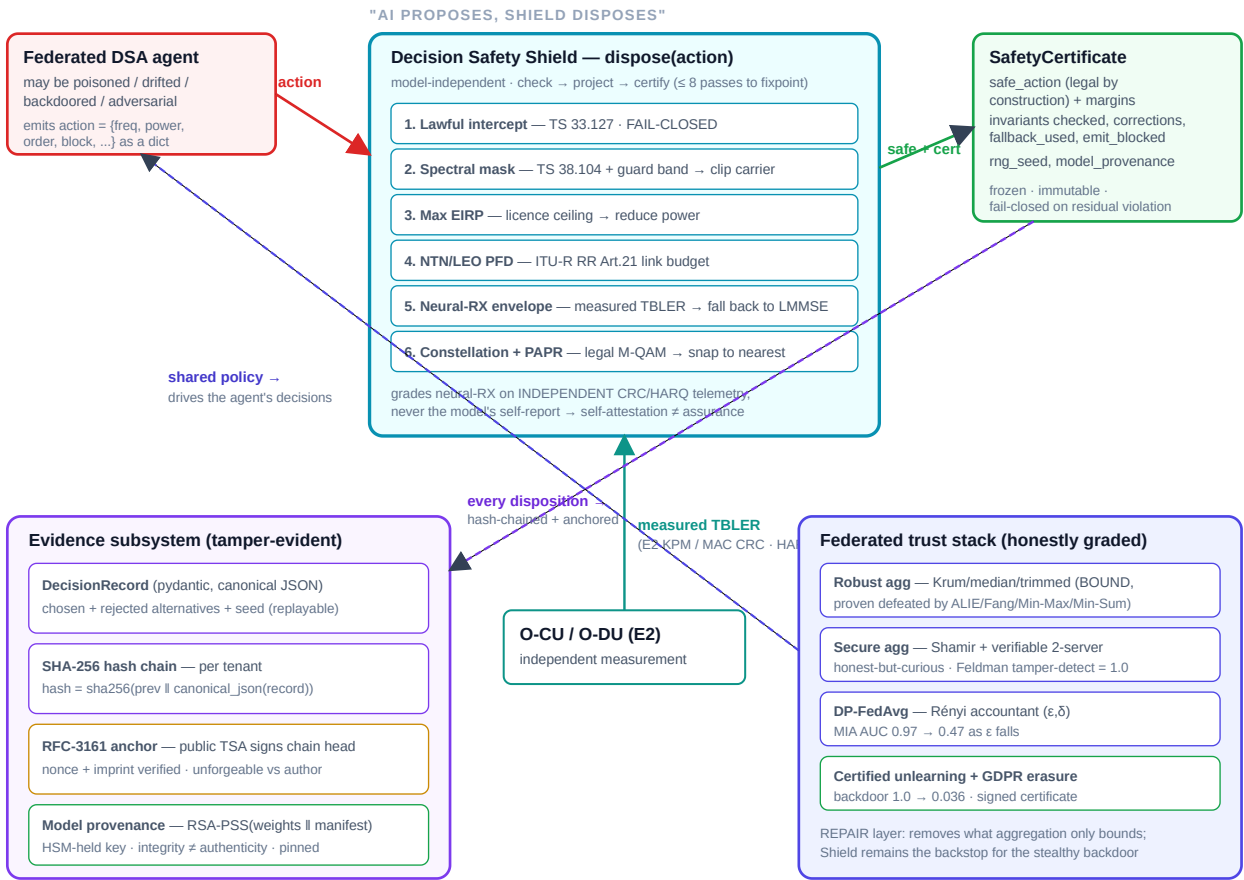
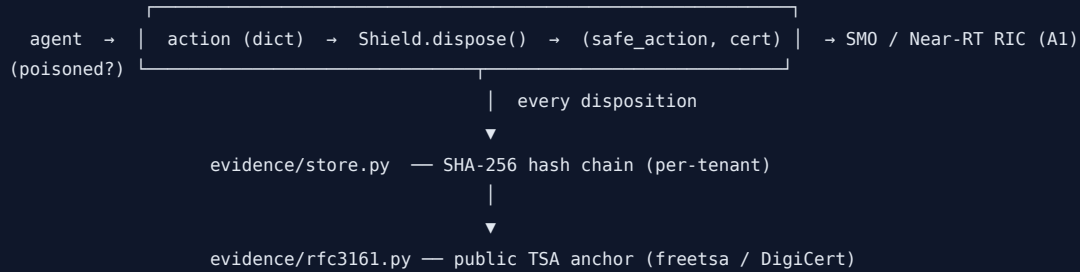


Figure 2. Internal architecture and data flow — the "AI proposes, Shield disposes" pipeline. A possibly-poisoned agent emits an action dict; the model-independent Shield projects it onto the ordered invariant chain (grading the neural receiver on *independent* CRC/HARQ telemetry); every disposition

yields an immutable SafetyCertificate that is hash-chained, RFC-3161-anchored, and provenance-bound. The federated trust stack (honestly graded, with certified unlearning as the repair layer) produces the shared policy that drives the agent.



4. System architecture



The federated side is a layered, **honestly-graded** defence-in-depth stack whose dependency spine is:

```
dsa_env → federated_q → {attacks, data_poison} → {poison_attacks, unlearning, erasure} → pipeline (Shield + evidence)
```

Each module's docstring names the exact threat it does **not** cover and points at the module that does. The load-bearing invariant of the whole system is stated in `spectrum/pipeline.py:16-18`: the deterministic Shield sits downstream of and independent from the ML policy, so poisoning/backdoor/DP-noise can degrade *throughput* but "*can never emit an illegal RF action.*"

5. The Decision Safety Shield

`src/horizon_ric/shield/` — the runtime-assurance core. Contract (`shield.py:1-16`): `safe, certificate = shield.dispose(action_ai)`.

5.1 The check → project → certify fixpoint loop

The Shield runs every invariant's `evaluate`, and if any is unsatisfied applies **every** invariant's `project`, iterating to a fixpoint (`shield.py:80-99`):

```
for _ in range(self.cfg.max_passes):          # max_passes = 8
    checks = self._evaluate(safe, context)
    unsatisfied = [c for c in checks if not c.satisfied]
    if not unsatisfied:
        break
    projected = True
    for inv in self._invariants:
        safe, corr = inv.project(safe, context)
        corrections.extend(corr)
    if safe.get("emit_blocked"):
        break
```

Iteration is *required* because one projection can break another invariant (e.g. the PFD-ceiling projection cuts Tx power, which changes the EIRP margin; a neural-RX fallback rewrites the `block` field). The loop re-evaluates until the action is stable or refused.

5.2 Fail-closed determination

Any residual violation after `max_passes` forces refusal (`shield.py:92-98`):

```
final_checks = self._evaluate(safe, context)
violated_ids = [c.invariant_id for c in final_checks if not c.satisfied]
emit_blocked = bool(safe.get("emit_blocked")) or bool(violated_ids)
...
if emit_blocked and not safe.get("emit_blocked"):
    safe["emit_blocked"] = True    # residual hard violation → refuse to emit
```

5.3 The invariant chain

`default_terrestrial_shield` (`shield.py:159-171`) orders the chain deliberately — **lawful intercept first** (legally mandatory, fail-closed), then spectrum/power physics, then AI-PHY envelopes:

Invariant	Basis	Action on violation
Lawful intercept	3GPP TS 33.127, fail-closed	refuse emit if LI scope unknown
Spectral mask	TS 38.104 §6.6 (+ guard band)	clip carrier inside licensed channel
Max EIRP	block-edge / licence limit	reduce power to ceiling
NTN/LEO PFD	ITU-R RR Art. 21 surface PFD	reduce satellite-link power
Neural-RX envelope	measured TBLER vs LMMSE baseline	fall back to certified classical RX
Constellation legality + PAPR	legal M-QAM orders, PAPR ceiling	snap to nearest legal order

The PFD physics is a genuine link-budget, not a constant (`invariants.py:215-226`): $PFD = EIRP_{dBW} - 10 \cdot \log_{10}(4\pi d^2) - 10 \cdot \log_{10}(BW_{MHz})$ with a default LEO slant range of ~550 km — a **distinct** invariant from EIRP, because a satellite link can satisfy the terrestrial EIRP ceiling yet still breach the surface PFD mask.

5.4 The security-critical design decision: *self-attestation is not assurance*

The single most important line of reasoning in the codebase lives in `NeuralRxEnvelopeInvariant` (`invariants.py:270-301`). An earlier version graded the neural receiver on its **self-reported** `predicted_tbler` — "exactly the field a poisoned model controls." A backdoored receiver could report a great TBLER and walk straight through the gate. The fix grades against an **independently-measured** `context["measured_tbler"]` (real CRC/HARQ telemetry from the MAC, which the receiver cannot forge). The envelope test (`invariants.py:340-344`):

```
ratio_dB = 10.0 * math.log10((base + eps) / (measured_val + eps))
margin_dB = ratio_dB + self.tolerance_dB
conf = float(action.get("demap_confidence", 1.0))
ok = (margin_dB >= 0.0) and (conf >= self.min_confidence)
```

If no independent measurement is present and `require_measurement=True` (default), the decision is **UNVERIFIED** and fails closed to the classical receiver (`invariants.py:317-332`). This is the difference between a security control and a checkbox.

Design rationale — why AI-PHY invariants *fall back* rather than *clip*. Physics/regulatory violations are clipped because the corrected action is still meaningful (an over-power carrier reduced to the ceiling is a valid carrier). An AI-PHY violation instead **swaps in the certified classical baseline** (LMMSE / classical QAM) because a misbehaving neural block cannot be trusted at all — you replace it, you do not tune it (`invariants.py:9-13`).

6. The evidence subsystem

`src/horizon_ric/evidence/` — tamper-evident, replayable, cryptographically time-anchored decision records.

6.1 The hash chain

`sha256_curr = sha256(prev_sha256 || canonical_json(record))` (`store.py:75-79`), first record chained from the all-zero hash. Verification (`store.py:117-139`) recomputes each link and returns the first broken index — any tamper of an older line changes its hash and invalidates every later `prev`. Canonicalisation (`store.py:68-72`) round-trips pydantic's JSON through `json.loads` and re-dumps with `sort_keys=True`, `separators=(",","")`, because verification must re-derive *identical bytes*.

6.2 Per-tenant chains and a documented concurrency fix

Chains are **per-tenant** (`verify_tenant`, `store.py:141-160`) so a tamper of tenant A cannot break tenant B's verification, and `__iter__` filters to the active `TenantScope` — the mechanism that stops `auditor@tenant_A` reading `tenant_B`. The `append` critical section is guarded by a lock (`store.py:203-228`) with a comment tracing it to a real reproduced bug (`test_known_bugs::CRIT-01`): without the lock, two threads read the same `prev` and write rows with identical `prev`, breaking the chain at the second row.

6.3 RFC-3161 anchoring — the unforgeability upgrade

A bare hash chain is *not* tamper-evident against the chain's own author (who can recompute the whole chain). `evidence/rfc3161.py` closes this by having **real public TSAs** (`freetsa.org` , `timestamp.digicert.com`) sign the chain head. The module header states the **anchored-chain unforgeability theorem** (`rfc3161.py:9-13`): under ROM on SHA-256 and EUF-CMA on TSA keys, any adversary mutating a record must either compromise $\geq M$ independent TSAs or break SHA-256 collision resistance.

The request path performs three integrity checks (`rfc3161.py:149-190`): `status == 0` , the returned **message-imprint equals our chain head**, and the **nonce round-trips** (offline-verifiable replay protection). The design refuses to hand-roll ASN.1 — it raises `ImportError` rather than fall back to a `pyasn1` path, because that "introduces another attack surface" (`rfc3161.py:28-34`). The anchor is itself a chain record, so tampering with it *also* breaks the SHA-256 chain.

6.4 Model provenance — integrity $\neq$ authenticity

`provenance/signing.py` signs `f"{version}|{weights_sha}|{manifest_sha}|{trainer_id}"` with an HSM-held RSA-PSS key (`signing.py:44-45` , `82-103`). Verification (`signing.py:126-149`) checks **both** that the weights hash matches (integrity) **and** that the embedded DER public key matches a pinned trusted key (authenticity) before verifying the PSS signature — defeating a self-signed model swap. The rationale (`signing.py:4-7`): a content-addressed hash proves *byte identity* but not *trust* — "a poisoned-but-consistent `.pt` passes a hash check." The provenance hash then threads into `SafetyCertificate.model_provenance` , unifying model authenticity with per-decision assurance.

6.5 Replayable counterfactuals — deliberately no LLM

`policy/counterfactual.py` materialises the top-K rejected alternatives, each stamped with the `random_seed` , so a regulator handed (`state` , `action_space` , `planner_config` , `seed`) can reconstruct the identical chosen action *and* rejected list. Human-readable explanations (`evidence/explanation.py:6-8`) use static templates, **not** an LLM, "because that would itself need certification" — a certifiable audit trail cannot depend on an uncertified generative component.

7. The federated trust stack

`src/horizon_ric/federated/` + `src/horizon_ric/spectrum/` . The learning problem is a torch-free **tabular federated Q-learning Dynamic Spectrum Access (DSA)** agent over a multi-user Gilbert–Elliott channel (`spectrum/dsa_env.py`). The trust machinery wraps it.

7.1 Robust aggregation — baseline, and *proven* defeated

`federated/robust.py` implements FedAvg, coordinate-median, trimmed-mean, and Krum (Blanchard et al. NeurIPS-17). Krum's score is the sum of the `m = n-f-2` smallest squared distances excluding self (`robust.py:112-123`). The repository is emphatic that these are **baseline, known-defeated** methods — and it *demonstrates* the defeat with real literature attacks in `spectrum/attacks.py` and `federated/poison_attacks.py` :

- **ALIE** ("A Little Is Enough", Baruch et al. NeurIPS-19) — each malicious update is $\mu - z \cdot \sigma$ coordinate-wise, a coordinated shift *inside* the benign variance envelope so order-statistic screens cannot distinguish it. The z-factor uses Acklam's inverse-normal approximation (`attacks.py:71-77`).
- **Fang** (USENIX Sec-20) — a directed deviation with a line-search that halves the magnitude until the crafted point would be *Krum-selected* (`attacks.py:129-139`).
- **Min-Max / Min-Sum** (Shejwalkar & Houmansadr NDSS-21) — binary-search the budget so the malicious point stays inside the benign ball (Min-Max) or satisfies a sum-of-squared-distances constraint that **matches Krum's own neighbour score** (Min-Sum, `poison_attacks.py:266-268`) — i.e. tuned to slip past Krum's selection.

The empirical punchline (reproduced live in §12): under adaptive Fang-median, the robust aggregators are *comparable-or-worse* than FedAvg, and **Krum's baseline distance is large even with no attack** (≈ 12.2) because it returns one client's update, not an average. Robust aggregation is a *bound*, not a cure. This is the opposite of the usual over-claim, and it is backed by a committed **five-family adversarial campaign** (75 adversarial tests).

7.2 The two documented tensions

- **Secure-agg $\perp$ robustness.** `federated/secure.py` implements (`t,n`) Shamir secret sharing over the Mersenne prime $2^{127}-1$ (`secure.py:22,45-72`), exploiting additive homomorphism so summing shares yields a share of the sum — but it is **honest-but-curious only** (Bonawitz et al. CCS-17). Crucially, secure aggregation is only wired for the mean family, because "*only the mean-family aggregators are additively homomorphic; Krum is not*" (`federated_q.py:122-130`). Secure aggregation hides exactly the per-client information a robust aggregator needs.

- **Malicious server.** `federated/verifiable_secagg.py` upgrades to a **two-non-colluding-server** model (NTU/DTC *Starfish*, arXiv:2404.09724): 2-of-2 additive sharing + **Feldman commitments** over the RFC-3526 2048-bit MODP prime. The homomorphic check $g^{\sum x} \stackrel{?}{=} \prod_i \text{Commit}_i$ per coordinate (`verifiable_secagg.py:145-153`) detects a server that drops or tampers a share. Measured tamper-detection and drop-detection rates are **1.0** over 200 trials (`results/verifiable_secagg.json`), and a single server's share view is chi-square-indistinguishable from a one-time pad ($\chi^2 z = 1.65$ over 65 536 bins). Honest scope: privacy holds **only under non-collusion**, and Feldman commitments are *binding, not hiding*.

7.3 Differential privacy with a real Rényi accountant

`federated/dp.py` implements DP-FedAvg (per-client L2 clip + Gaussian mechanism) and a genuine Rényi-DP accountant (Mironov CSF-17). The RDP $\rightarrow (\epsilon, \delta)$ conversion minimises $\epsilon_{\text{RDP}}(\alpha) + \log(1/\delta)/(\alpha-1)$ over an order grid (`dp.py:67-75`); RDP composes additively (`dp.py:94-99`). The reported privacy/utility curve is honest about the cost — the clip norm is data-dependent (median member-update norm, "a mild DP-leak, reported not hidden"). Measured (§12): membership-inference AUC falls from **0.97** $\rightarrow$ **0.47** as ϵ falls from $\infty \rightarrow 0.62$, with throughput falling in lock-step — the real privacy/utility frontier, not a cherry-picked point.

7.4 Certified unlearning and GDPR erasure — the *repair* layer

Robust aggregation only *bounds* a poisoner; `federated/unlearning.py` **removes** an attributed client's contribution and binds the removal to a signed `UnlearningCertificate` (distance-to-retrain bound + backdoor-success probe). The module is scrupulously honest about its own limits (`unlearning.py:214-217`): the whole-vector outlier detector *cannot* see a **sparse, norm-matched backdoor** — which is precisely why that backdoor survives robust aggregation in the first place — and there "the deterministic Shield remains the backstop." Measured (§12): under undefended FedAvg a backdoor lands at success **1.0**; from-scratch certified unlearning drives it to the **clean floor** $\approx$ **0.036** with certified L2 distance ≈ 0 to the gold-standard retrain, and the certificate *rejects* the still-poisoned model.

`federated/erasure.py` goes finer — **subject-level GDPR Art. 17 erasure**. By making training *transition-based and subject-tagged*, erasure becomes an exact recompute with the subject's rows filtered out (`erasure.py:201-209`), with **no cross-round warm-starting**, so `global_after` equals a genuinely independent from-scratch retrain for *any* aggregator $\Rightarrow$ certified distance **exactly 0** — provable, not approximate erasure.

8. The physical-layer adversarial testbed

`src/horizon_ric/phy/` is a self-contained, torch-free adversarial-ML testbed whose purpose is to face the Shield with attacks it was **not hand-coded against**.

8.1 The hand-derived neural receiver and its verified gradient

`neural_rx.py` is a 2-layer numpy MLP (He init) with hand-written forward, backprop, and — the load-bearing quantity — the **input gradient** that PGD ascends (`neural_rx.py:102-108`):

```
z1, h, z2, p = self._forward(X)
dz2 = p.copy()
dz2[np.arange(len(X)), y] -= 1.0 # softmax-CE gradient (un-normalised)
dh = dz2 @ self.W2.T
dz1 = dh * (z1 > 0) # ReLU mask
dX = dz1 @ self.W1.T
return dX
```

8.2 The router-not-denoiser architecture

The certified fallback is the classical ML demapper — nearest-constellation-point (`constellation.py:39-44`). Its Voronoi boundaries are straight and max-margin, so "a small bounded perturbation below the margin cannot flip it. That margin property is exactly why it is the safe fallback" (`constellation.py:4-7`). PGD (`pgd.py:50-57`) is standard L_∞ projected ascent; the fading attack respects a `perturb_mask = [1, 1, 0, 0]` so the attacker perturbs the received signal Y but **not** the channel estimate H — a threat-model contract encoded in code. The full battery (`evasion.py`) adds FGSM/BIM/MIM plus **true black-box** transfer (crafted on a differently-sized surrogate, target never queried for gradients) and boundary (hard-label only) attacks, and reports *honestly* where the gap collapses (fading; black-box weaker than white-box).

The `jamming.py` module is where the honesty is sharpest: under **barrage jamming** both receivers degrade identically, so the router provably cannot help — "the Shield is a router, not a denoiser; it cannot restore a noisier channel." The committed jamming results (§12) show exactly this: fallback rate 0.0 and shield-effective SER = neural SER under barrage.

9. O-RAN adapters and runtime resilience

Multi-dialect A1 (`rapp/a1_adapter.py`). The same logical A1 policy operation maps to four vendor wire surfaces via a `dialect` switch — `legacy`, `osc` (OSC PMS, flat PUT), `eiap` (Ericsson EIAP, camelCase), `mantaray` (Nokia SDN-R). Each URL builder and body serialiser branches on dialect (`a1_adapter.py:185-320`). Evidence is persisted **only after** the PUT succeeds and under an `asyncio.Lock` (`a1_adapter.py:322-341`), so "audit trails only contain policies the Near-RT RIC actually accepted." Schemas are fail-closed: unknown policy types are *refused* rather than shipped with a permissive schema (`a1_adapter.py:662-663`). R1 (`r1_adapter.py`) and O1 (`o1_adapter.py` , async wrapper over `ncclient` `NETCONF`) follow the same real-wire-behind-a-breaker pattern.

Atomic A → B promotion (`runtime/atomic_promotion.py`). A single-pointer atomic swap (`atomic_promotion.py:150-159`) with SHA-keyed reader release (`atomic_promotion.py:106-118`) and cooperative draining of the old slot until `in_flight ≤ 0` — so no in-flight decision uses a half-loaded model. `rollback()` restores the exact prior `_ActiveSlot` object (same SHA), giving **bit-identical** rollback rather than a reload.

Circuit breaker (`runtime/circuit_breaker.py`). Drives the pybreaker state machine directly (avoiding pybreaker's Tornado-dependent async surface). The key policy: **5xx and timeouts trip the breaker; 4xx do not** (`circuit_breaker.py:136-157`), because a 4xx means the client erred but the server is healthy. Every transition emits both a structlog event and an X.733 OAM alarm.

Graceful degradation (`runtime/graceful_degradation.py`). A fixed failure-mode → degraded-state FSM (SMO unreachable → replay last-good; evidence store down → buffer to `/tmp`; telemetry dropped → hold output). `is_serving()` directly drives `/readyz` → 503 in any degraded mode.

10. Security subsystem

- **HSM (`security/hsm.py`).** A PKCS#11 abstraction with an in-memory RSA-2048 test backend and a real SoftHSM2 backend; production CloudHSM/Luna reuse the same PKCS#11 path. Signing is RSA-PSS/SHA-256 (`hsm.py:178-187`); handles are side-typed so a public handle can never sign. Share-dealer keys live in the HSM, not process memory.
 - **RBAC (`security/rbac.py`).** Casbin "RBAC with domains" (five roles per tenant), with `keyMatch` wildcards and a **fail-closed production guard** that refuses to run with only the `default` domain (`rbac.py:178-192`). Policy completeness is *SMT-proven* with z3 in the test suite.
 - **JWT (`security/jwt.py`).** RS256 with zero-downtime key rotation and a **stated formal bound**: under EUF-CMA and bounded skew, a token minted with the old key cannot verify after $T_{rot} + W + \Delta_{skew}$. The mechanism is `_prune_retired_keys` called eagerly at every verify (`jwt.py:240-244`), with operator sizing $W \geq T_{ttl} + \Delta_{skew}$ (defaults $W = 24\text{ h}$, $\Delta_{skew} = 60\text{ s}$).
 - **NIS2 (`security/nis2_reporter.py`).** Implements Directive (EU) 2022/2555 Art. 23(4) 24 h / 72 h / 30 d clocks, with three automatic triggers — the most telling being `auto_check_audit_chain`, which fires a *critical* incident when `store.verify()` finds a broken hash-chain index. Delivery failures are persisted to the audit chain rather than dropped.
-

11. Test methodology — how the tests prove the claims

The `tests/` suite is not smoke coverage; it is **adversarial evidence**. Below are the representative patterns, each with the reasoning for *why* it is designed that way.

11.1 Proving an attack is *real* — the finite-difference gradient check

The strongest claim in the PHY testbed is "we mount a *genuine* white-box PGD attack." That claim is only true if the input gradient is truly the loss gradient. `tests/test_neural_rx_pgd.py:25-47` proves it by checking the hand-derived backprop against a central-difference estimate:

```
g = net.input_gradient(Xs, ys)          # analytic
...
num[i, j] = (lp - lm) / (2 * eps)        # numerical, eps = 1e-5
rel = np.abs(g - num).max() / (np.abs(num).max() + 1e-9)
assert rel < 1e-4, f"input gradient does not match finite difference (rel={rel:.2e})"
```

Why coded this way: it forecloses the reviewer objection that the adversarial result is a framework artefact. Live result (this environment): `tests/test_neural_rx_pgd.py ... 4 passed in 6.78s`.

11.2 Proving the Shield holds under a *poisoned* model — property assertions

`tests/test_shield.py:44-76` constructs a maximally-poisoned action (out-of-band frequency, 46 dBm EIRP over a 33 dBm ceiling, illegal 1024-QAM, over-PAPR) and asserts the disposed action is legal on **every** axis:

```
assert 3.40e9 <= a["frequency_hz"] <= 3.50e9
assert a["tx_power_dBm"] + a["antenna_gain_dBi"] <= 33.0 + 1e-6
assert a["constellation_order"] in (4, 16, 64, 256)
assert a["papr_dB"] <= 8.5 + 1e-6
```

Why coded this way: it tests the *guarantee* ("a poisoned model cannot emit illegally"), not a happy path — the test is the executable form of the security claim.

11.3 Proving tamper-evidence — reconstructing the attack

`tests/test_integrity_attacks.py` (and its benchmark) mount **8 integrity probes** — model swap, weight-byte tamper, manifest tamper, evidence-chain field tamper, reorder/replay, self-report spoof, illegal-emit, LI fail-closed bypass — and assert each is detected or blocked. Live result: **8/8 blocked**, each with the exact defending mechanism named (e.g. probe 6 blocked by "grades on independent measured_tbler, not self-report").

11.4 Proving robustness claims are *honest* — tests that assert a defence *fails*

`tests/test_fl_poison_battery.py` and the committed `results/fl_poisoning_suite.json` sweep 9 Byzantine fractions × 5 attack families × 8 seeds and record where the aggregators *lose*. This is a rare and valuable test-design choice: the suite is engineered to surface the breakdown, not hide it.

11.5 Proving concurrency correctness — regression tests for named bugs

`tests/test_known_bugs.py` encodes reproducers for specific historical defects (`CRIT-01` , `CRIT-02` , `HIGH-01`) — e.g. the evidence-chain race that the `append` lock fixes. The source comments cite these IDs directly, so the fix and its proof are permanently coupled.

11.6 Proving policy completeness — an SMT proof, not a sample

`tests/test_rbac_smt_completeness.py` uses the z3 solver to prove the Casbin policy has no reachable (subject, object, action) gap — a formal-methods test, not example-based coverage.

Aggregate result (measured, this environment):

```
pytest tests/ -m "not integration and not slow"
490 passed, 3 skipped, 1 xfailed          (exit code 0)
```

12. Consolidated empirical results

All numbers below were **reproduced live** in this environment (commands in §14) and cross-checked against the committed `benchmarks/results/*.json`.

12.1 The Shield on a poisoned decision stream (headline)

`poisoning_shield_benchmark.py --decisions 10000 --poison-rate 0.30`, graded by an **independent** emission-mask / ACLR oracle (not the Shield's own constants):

Metric	Unguarded	With Shield
Out-of-spec emits on the air interface	1,983	0
In-spec-but-harmful emits	489	0
Decisions carrying a certificate	0	10,000

Live re-run matched the committed JSON exactly.

12.2 Robust aggregation — the honest federated picture

16 honest / 4 Byzantine (20%), dim 200. Distance from the honest mean (**lower is better**); note Krum's large no-attack baseline:

Attack	FedAvg	Krum	Median	Trimmed-mean
No attack (baseline)	0.00	12.20	2.47	0.61
ALIE (z = 0.157)	0.43	2.16	1.99	1.30
Fang-Krum	1.77	8.84	4.82	3.05
Fang-median	8.23	12.20	4.96	5.81

Interpretation (from the benchmark's own `honest_findings`): no robust aggregator is a silver bullet — they *bound* the damage, they do not eliminate it, and under adaptive attacks Krum is often the worst.

12.3 Certified unlearning repairs what aggregation leaks

10 clients, 1 malicious, backdoor forcing a channel the healthy policy avoids:

Model state	Backdoor success	Certified L2 to retrain
Poisoned (undefended FedAvg)	1.000	—
Cheap efficient-unlearn	1.000 (insufficient — flagged)	13.99 (large → rejected)
From-scratch certified unlearn	0.036 (clean floor)	≈ 0.000

The certificate *verifies* against the unlearned model and *rejects* the poisoned one — the honesty property: it never claims a removal that did not happen.

12.4 Differential privacy — the measured privacy/utility frontier

DP-FedAvg + Rényi accountant, 8 members / 8 non-members, $\delta = 1e-5$:

Noise z	ϵ (measured)	MIA AUC	Throughput/slot
0.0	∞ (no DP)	0.969	1.085
1.0	5.303	0.688	0.480
4.0	1.268	0.604	0.132
8.0	0.621	0.469	0.047

Membership inference collapses toward chance (0.5) as ϵ shrinks — at real utility cost, reported not hidden.

12.5 Adversarial PHY — the Shield as a router

16-QAM, 22 dB, PGD $\epsilon = 0.12$ (`results/neural_rx_pgd.json`):

	Clean SER	PGD SER	PGD TBLER
Neural RX	0.000	0.0147	0.7625
Classical demapper	0.000	0.00068 (~22× lower)	0.065
Shield-effective	—	—	0.065 (fallback rate 1.0, ~12× TBLER reduction)

12.6 Integrity and verifiable secure aggregation

- Integrity attack battery: **8/8 probes blocked** (live).
- Verifiable 2-server secure-agg: tamper-detection and drop-detection rates **1.0** over 200 trials; single-server share view ≈ one-time pad (χ^2 z = 1.65).

13. Honest limitations

The repository states these plainly; a faithful report must repeat them.

1. **Aggregator robustness is baseline only** — Krum/median/trimmed-mean are defeated by adaptive ALIE/Fang/Min-Max/Min-Sum, and the codebase *proves* this with a 75-test adversarial campaign. The guarantee is the deterministic Shield, with certified unlearning as the repair layer.
 2. **Secure aggregation is honest-but-curious** and in tension with robustness; the verifiable two-server scheme relies on **non-collusion**.
 3. **Invariant coverage is enumerated, not complete** — behaviours not expressible as a listed invariant are outside the guarantee.
 4. **Band numbers are calibrated per-deployment, not byte-verified** against TS 38.104 in this repo.
 5. **The stealthy, norm-matched backdoor** evades whole-vector attribution — the regime where the Shield backstop, not unlearning, is what holds.
 6. **The federated-DSA demonstrator, dataset, and LEO-PFD demonstrator** are an in-build work-stream (M1–2); the trust mechanisms they depend on are working, tested code.
-

14. Reproduction appendix

```
# 1. Clean environment (the stock image ships a Debian-managed cryptography)
python3 -m venv .venv && . .venv/bin/activate
pip install -e ".[dev,otel,oran,persistence]"

# 2. Full fast test pack → 490 passed, 3 skipped, 1 xfail, 0 failed
pytest tests/ -m "not integration and not slow" -q

# 3. Headline Shield benchmark (deterministic – matches committed JSON exactly)
python benchmarks/poisoning_shield_benchmark.py --decisions 10000 --poison-rate 0.30

# 4. Federated + PHY + security benchmarks
python benchmarks/federated_unlearning_suite.py
python benchmarks/dp_privacy_suite.py
python benchmarks/integrity_attack_suite.py          # 8/8 blocked
python benchmarks/verifiable_secagg_suite.py
python benchmarks/neural_rx_pgd_benchmark.py

# 5. The gradient-authenticity proof (the attack is real)
pytest tests/test_neural_rx_pgd.py -v                # 4 passed
```

Verification of determinism (observed). The live `poisoning_shield_benchmark` run printed `krum 12.196867167126614`, `fang_median fedavg 8.233744329869157`, and `1983 illegal emits → 0`, matching `benchmarks/results/poisoning_shield.json` field-for-field — the property that makes these results auditable.

Prepared as an independent technical review of the Horizon-RIC repository. All quantitative results were reproduced from a clean install; file:line citations refer to the committed source at the time of writing.